

**Security Review: EUROGRANT**

## Scope

## Scan Summary

## Threat Model

## Assets And Boundaries

## Required Security Invariants

## Realistic Attackers

## Findings

## Confidence Scale

- [1] Existing-tenant enrollment bypass through substring domain matching
- [2] Inactive users remain fully authenticated
- [3] Viewer can modify organization-wide settings
- [4] DOCX archive expansion is not bounded before parsing
- [5] Production Redis defaults do not require authentication or TLS

# Security Review: EUROGRANT

## Scope

- Scan mode: repository-wide security review at commit `aa04a92`.
- In scope: FastAPI backend, Next.js frontend, Nginx, Docker/Compose, workers, storage and AI integrations, migrations, dependency manifests, tests, and CI configuration.
- Coverage: 95 ranked source/configuration files received full-file review receipts; 29 security surfaces were reconciled.
- Context: the threat model was generated during Phase 1 from repository evidence, not supplied as an external assessment.
- Validation: focused static traces, bounded FastAPI reproductions, a production Next.js build, dependency audits, Bandit, tests, lint, and build checks.
- Limitations: Docker runtime execution and a destructive parser-exhaustion proof were not performed. Deployment-specific network exposure cannot be proven from this repository alone.
- Repository integrity: no source files were changed by this scan. The existing untracked `CON` path was left untouched.

## Scan Summary

| Field               | Value                                                                                         |
|---------------------|-----------------------------------------------------------------------------------------------|
| Reportable findings | 6                                                                                             |
| Severity mix        | 2 high, 1 medium, 3 low                                                                       |
| Confidence mix      | 4 high, 1 medium-high, 1 medium                                                               |
| Candidate closure   | 12/12 discovery, validation, and attack-path receipts                                         |
| File coverage       | 95/95 ranked files completed                                                                  |
| Validation mode     | Runtime reproduction, production build evidence, static trace, tests, and dependency scanners |

Artifacts:

- Threat model: `artifacts/01_context/threat_model.md`
- Discovery report: `artifacts/02_discovery/finding_discovery_report.md`
- Coverage ledger: `artifacts/03_coverage/repository_coverage_ledger.md`
- Validation summary: `artifacts/05_findings/validation_summary.md`
- Attack-path summary: `artifacts/05_findings/attack_path_analysis_report.md`

## Threat Model

EUROGRANT is a multi-tenant grant-discovery and proposal application. Its public runtime comprises a Next.js frontend, FastAPI backend, PostgreSQL, Redis/Celery workers, optional S3 storage, Pinecone, AI providers, and Nginx.

## Assets And Boundaries

- Credentials, JWT sessions, invite codes, roles, organization membership, uploaded business documents, proposals, and third-party service credentials are sensitive assets.
- Internet requests, browser API calls, uploaded files, external grant content, parser input, provider responses, and deployment configuration cross trust boundaries.
- Authenticated tenant users remain potentially malicious. Server-side organization scoping and role checks are required.
- Uploaded content remains untrusted after extension, MIME, and magic-byte checks.
- Secure production behavior must fail closed when infrastructure configuration is absent or unsafe.

## Required Security Invariants

- Protected operations authenticate a live user and derive tenant scope server-side.
- Privileged mutations enforce backend role checks.
- Registration cannot enroll an unauthorized user into an existing organization.
- Uploaded files cannot exhaust shared workers or escape storage boundaries.
- Errors and frontend bundles do not expose credentials or unsafe internal destinations.
- Production services use authenticated, least-privilege infrastructure connections.

## Realistic Attackers

Unauthenticated internet users, compromised or malicious tenant accounts, malicious document uploaders, and attackers controlling external content are in scope. Repository maintainers, physical host attackers, and already-compromised cloud operators are outside the direct application threat model.

## Findings

| # | Severity | Finding                                                                             |
|---|----------|-------------------------------------------------------------------------------------|
| 1 | High     | <a href="#">Existing-tenant enrollment bypass through substring domain matching</a> |
| 2 | High     | <a href="#">Inactive users remain fully authenticated</a>                           |
| 3 | Medium   | <a href="#">Viewer can modify organization-wide settings</a>                        |
| 4 | Low      | <a href="#">DOCX archive expansion is not bounded before parsing</a>                |
| 5 | Low      | <a href="#">Production Redis defaults do not require authentication or TLS</a>      |
| 6 | Low      | <a href="#">Login lockout enables targeted account denial</a>                       |

## Confidence Scale

| Label | Meaning                                                                                                     |
|-------|-------------------------------------------------------------------------------------------------------------|
| high  | Direct source, configuration, or runtime evidence supports the finding with no material unresolved blocker. |

| Label  | Meaning                                                                                      |
|--------|----------------------------------------------------------------------------------------------|
| medium | Source evidence supports the issue, but deployment or exploit reliability still needs proof. |
| low    | Weak or incomplete evidence; retained only as an explicit follow-up candidate.               |

## [1] Existing-tenant enrollment bypass through substring domain matching

|                             |                                                                                                                                                                        |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Severity</b>             | High                                                                                                                                                                   |
| <b>Confidence</b>           | High                                                                                                                                                                   |
| <b>Confidence rationale</b> | A bounded end-to-end FastAPI reproduction enrolled a lookalike-domain attacker into an existing tenant and proved tenant-scoped reads.                                 |
| <b>Category</b>             | Authorization bypass / tenant isolation failure                                                                                                                        |
| <b>CWE</b>                  | CWE-863: Incorrect Authorization                                                                                                                                       |
| <b>Affected lines</b>       | <pre> backend/app/routers/auth.py:37-52 , backend/app/routers/auth.py:58-75 , backend/app/routers/organizations.py:10-22 , backend/app/routers/uploads.py:63-73 </pre> |

### Summary

Existing-organization registration checks whether the normalized organization name is a substring of the email domain, rather than verifying an approved exact domain or tenant-specific invitation. A domain such as `evilacme.com` therefore passes for organization `Acme`, and the created viewer account is bound to the victim organization's ID.

### Validation

The reproduction created an `Acme` owner, rejected `attacker@evilcorp.com`, then accepted `attacker@evilacme.com` with the same organization ID. The attacker logged in and read the organization profile and seeded victim document metadata. The attacker still needs the shared invite code and exact organization name, and

receives viewer rather than writer/admin privileges.

**Dataflow**

Public registration JSON -> exact organization-name lookup -> substring domain comparison -> user creation with victim `organization_id` -> JWT login -> organization-scoped profile and document readers.

**Reachability**

The entry point is public and remote. Preconditions are knowledge of the shared invite code and exact organization name plus control of a lookalike email domain. The attack crosses the tenant boundary and grants persistent viewer access.

**Severity**

Final severity is high because runtime evidence proves a realistic cross-tenant authorization bypass and access to meaningful tenant metadata. It is not critical because the path requires a shared invite code and grants viewer-only access. Evidence that the invite code is tightly single-use would lower severity; proof of sensitive document or proposal content exposure would increase impact.

**Remediation**

Replace substring matching with explicit tenant-approved domains or tenant-scoped, single-use invitations. Normalize domains using a domain parser, require administrative approval for existing-tenant enrollment, and add regression tests for prefix, suffix, Unicode, and lookalike domains.

**[2] Inactive users remain fully authenticated**

|            |      |
|------------|------|
| Severity   | High |
| Confidence | High |
|            |      |

|                             |                                                                                                                                  |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Confidence rationale</b> | HTTPS TestClient reproduction proved an inactive user can log in, receive a secure session cookie, and access a protected route. |
| <b>Category</b>             | Authentication bypass / account suspension failure                                                                               |
| <b>CWE</b>                  | CWE-287: Improper Authentication                                                                                                 |
| <b>Affected lines</b>       | <code>backend/app/routers/auth.py:87-120</code> , <code>backend/app/auth.py:56-59</code> , <code>backend/app/models.py:50</code> |

## Summary

The user model contains `is_active` , but neither password login nor JWT-backed user restoration checks it. Marking an account inactive therefore does not revoke its ability to authenticate or use protected APIs.

## Validation

A scratch SQLite-backed FastAPI run created `is_active=False` credentials.

`POST /api/v1/auth/login` returned 200, and both secure-cookie and bearer-token flows accessed `GET /api/v1/organizations/me` successfully.

## Dataflow

Valid inactive credentials -> password verification without activity check ->

JWT issuance -> subject lookup without activity check -> protected tenant route.

## Reachability

A remote attacker needs valid credentials for an account administrators intended to disable. The attacker regains that account's existing tenant role and access across routes protected only by `get_current_user` .

## Severity

Final severity is high because this directly defeats an identity suspension control and preserves access to tenant data and actions. It is not critical

because it does not impersonate unrelated users or expand the disabled account's existing privilege scope. Evidence that inactive accounts are never used as a security control would lower severity; proof of privileged disabled accounts in production would increase urgency.

## Remediation

Reject inactive users before token issuance and in `get_current_user` so existing tokens stop working immediately. Add tests for login, cookie, bearer, role-gated, and background-task paths using inactive accounts.

## [3] Viewer can modify organization-wide settings

|                      |                                                                                                                                                 |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Severity             | Medium                                                                                                                                          |
| Confidence           | High                                                                                                                                            |
| Confidence rationale | A real viewer JWT persisted organization-wide threshold, notification, and profile changes through the public API.                              |
| Category             | Missing function-level authorization                                                                                                            |
| CWE                  | CWE-862: Missing Authorization                                                                                                                  |
| Affected lines       | <code>backend/app/routers/organizations.py:55-73</code> ,<br><code>backend/app/auth.py:62-71</code> , <code>backend/app/schemas.py:65-74</code> |

## Summary

`PUT /api/v1/organizations/me` authenticates the caller but does not require a writer/admin role. Its update schema exposes shared matching, notification, and organization-profile fields, all of which are persisted directly.

## Validation

A viewer JWT was confirmed through `/users/me` , then used to update

`match_threshold` , `alert_email_enabled` , and `sector` . The route returned 200 and

the database values changed. Nearby uploads demonstrate that the codebase already has a role dependency for privileged mutations.

**Dataflow**

Viewer JWT -> organization update JSON -> authentication only -> unrestricted schema field iteration -> `setattr` on shared organization -> database commit.

**Reachability**

Any authenticated viewer in a tenant can trigger the route remotely. The impact crosses from one low-privilege user to settings consumed by the entire tenant, but does not cross into another organization.

**Severity**

Final severity is medium: likelihood is high, while impact is limited to same-tenant integrity and workflow disruption rather than cross-tenant data access or account takeover. An explicit product contract granting viewers these mutations would invalidate the finding; evidence that the settings control security-critical notifications would increase impact.

**Remediation**

Require writer/admin authorization for shared settings, or split viewer-safe self-service fields from administrator-only organization fields. Add explicit viewer `403` tests for each protected field.

**[4] DOCX archive expansion is not bounded before parsing**

|                      |                                                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Severity             | Low                                                                                                                                                  |
| Confidence           | medium-high                                                                                                                                          |
| Confidence rationale | Static tracing proves attacker-controlled DOCX ZIP bytes reach <code>python-docx</code> without expansion controls; a destructive exhaustion PoC was |



|                       |                                                                                                                   |
|-----------------------|-------------------------------------------------------------------------------------------------------------------|
|                       | intentionally not run.                                                                                            |
| <b>Category</b>       | Uncontrolled resource consumption                                                                                 |
| <b>CWE</b>            | CWE-409: Improper Handling of Highly Compressed Data                                                              |
| <b>Affected lines</b> | backend/app/routers/uploads.py:76-123 ,<br>backend/app/services/extraction.py:53-56 ,<br>backend/app/worker.py:82 |

## Summary

The upload route limits compressed size to 25 MB and checks superficial DOCX properties, but does not bound expanded size, member count, compression ratio, XML depth, or OOXML part sizes before `docx.Document` opens the ZIP container.

## Validation

Static source-to-parser tracing confirmed authenticated uploader control through storage retrieval and `BytesIO` into `python-docx`. Celery time limits, retries, container memory caps, rate limiting, and writer/admin upload authorization are material countercontrols. No destructive ZIP-bomb execution was performed.

## Dataflow

Authenticated DOCX upload -> compressed-size/MIME/magic checks -> object storage -> worker reads full bytes -> `docx.Document(BytesIO(...))` expands and parses archive.

## Reachability

A malicious upload-capable tenant user can submit the document remotely. The likely consequence is bounded worker memory/CPU pressure and repeated retries, not proven platform-wide outage.

## Severity

Final severity is low after mechanically combining medium impact and medium

likelihood. A bounded test proving shared-worker starvation would raise severity; evidence that the parser enforces strict expansion limits would close the issue.

**Remediation**

Inspect the ZIP central directory before parsing and reject excessive expanded size, member count, compression ratio, nesting, and per-part size. Parse in an isolated resource-constrained process, avoid retrying deterministic malformed files, and add safe compressed-fixture tests.

**[5] Production Redis defaults do not require authentication or TLS**

|                      |                                                                                                                                                                             |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Severity             | Low                                                                                                                                                                         |
| Confidence           | Medium                                                                                                                                                                      |
| Confidence rationale | Application defaults and consumers are directly evidenced, but production network reachability is not established and Compose supplies a password and loopback publication. |
| Category             | Insecure infrastructure default                                                                                                                                             |
| CWE                  | CWE-1188: Insecure Default Initialization of Resource                                                                                                                       |
| Affected lines       | backend/app/config.py:12-14 , backend/app/worker.py:13-17 , backend/app/services/lockout.py:13-18                                                                           |

**Summary**

Production is the default environment, while the default Celery broker, result backend, and lockout Redis URL use plaintext unauthenticated `redis://`. Startup does not reject these defaults in production.

**Validation**

The same default URL feeds Celery and account lockout. Network access to such an instance would allow queue or lockout-state manipulation. Checked-in Compose is important counterevidence: it adds a Redis password and limits host publication

to loopback, although the application defaults do not encode that authentication.

**Dataflow**

Missing production Redis environment value -> unauthenticated plaintext URL ->  
Celery broker/result backend and lockout client -> queue and authentication-defense state.

**Reachability**

Exploitation requires an unsafe deployment that makes the Redis service reachable from an untrusted network. The repository does not prove the real production network layout, so likelihood remains unknown.

**Severity**

Final severity is low because medium potential impact combined with unknown likelihood yields low under the policy matrix. Evidence of remotely reachable production Redis would increase severity; enforced authenticated TLS startup validation would close it.

**Remediation**

Remove functional production defaults. Require an authenticated `rediss://` URL or equivalent secret-backed configuration at startup, use separate least-privilege Redis roles/databases where supported, and add configuration tests that production fails closed.

**[6] Login lockout enables targeted account denial**

|                      |                                                                                                                           |
|----------------------|---------------------------------------------------------------------------------------------------------------------------|
| Severity             | Low                                                                                                                       |
| Confidence           | High                                                                                                                      |
| Confidence rationale | The public source, lockout ordering, five-attempt threshold, and route-rate interaction are directly established in code. |
| Category             | Authentication denial of service                                                                                          |

**CWE**

CWE-645: Overly Restrictive Account Lockout Mechanism

**Affected lines**`backend/app/routers/auth.py:89-104 ,  
backend/app/services/lockout.py:46-61`**Summary**

Five failed login attempts for an attacker-selected email lock that account for 30 minutes. Lockout is checked before a correct password, and the route's five-per-minute throttle still allows the complete lockout burst.

**Validation**

The code trace confirmed username-controlled counters, threshold, duration, and pre-password lockout ordering. Repeating the burst can maintain denial. Existing IP throttling limits broad abuse but does not protect one chosen account.

**Dataflow**

Public login email -> per-email failed-attempt counter -> fifth failure sets 30-minute lock -> later correct login rejected before password validation.

**Reachability**

An unauthenticated remote attacker needs a known user email and five requests from one permitted source. Impact is limited to transient denial for selected accounts.

**Severity**

Final severity is low because the impact is one-account, temporary availability loss and the IP throttle reduces abuse scale. Evidence of distributed bypass or administrative account targeting at scale would raise severity; user-confirmed unlock or risk-based throttling would reduce it.

**Remediation**

Avoid hard account locks driven solely by unauthenticated failures. Use exponential

delay, per-IP/device risk signals, user notifications, challenge flows, and secure self-service unlock. Test that an attacker cannot deny a correct login with a small unauthenticated burst.

## Reviewed Surfaces

| Surface                            | Risk Area                                 | Outcome                   | Notes                                                                                                         |
|------------------------------------|-------------------------------------------|---------------------------|---------------------------------------------------------------------------------------------------------------|
| Registration and tenant enrollment | Tenant isolation                          | Reported                  | SEC-001                                                                                                       |
| Login, JWT restoration, user state | Authentication lifecycle                  | Reported                  | SEC-002                                                                                                       |
| Organization settings mutation     | Role authorization                        | Reported                  | SEC-003                                                                                                       |
| Proposal generation by viewers     | Role/product policy                       | Needs follow-up           | SEC-004 lacks an authoritative role-permission contract.                                                      |
| DOCX parsing                       | Resource exhaustion                       | Reported                  | SEC-005                                                                                                       |
| PDF parsing                        | Resource exhaustion                       | Needs follow-up           | SEC-006 needs a bounded crafted-file reproduction.                                                            |
| Frontend API base URL              | Production correctness / local-host trust | Rejected as vulnerability | SEC-007 is a serious deployment defect, but the security exploit requires a victim-local compatible listener. |
| Redis/Celery/lockout configuration | Service authentication                    | Reported                  | SEC-008                                                                                                       |
| Login lockout                      | Targeted availability                     | Reported                  | SEC-009                                                                                                       |
| Health endpoint errors             | Information disclosure                    | Rejected                  | SEC-010 is low-value diagnostic leakage without a stronger demonstrated consequence.                          |

| Surface                             | Risk Area                                  | Outcome                   | Notes                                                                                               |
|-------------------------------------|--------------------------------------------|---------------------------|-----------------------------------------------------------------------------------------------------|
| CI audit gates                      | Build protection                           | Rejected as vulnerability | SEC-011 remains engineering process debt because scans are non-blocking.                            |
| Nested PostCSS advisory             | Dependency reachability                    | Rejected                  | SEC-012 has no attacker-controlled CSS path. Upgrade remains recommended.                           |
| SQLAlchemy query construction       | Injection / tenant scope                   | No issue found            | Bound expressions and organization filters were reviewed across routers/services.                   |
| Upload storage keys and downloads   | Path traversal / object access             | No issue found            | UUID keys and organization-prefixed lookups were present.                                           |
| AI and vector integrations          | Prompt injection / secret or tool crossing | No issue found            | No privileged tools, secret-return path, or cross-tenant retrieval chain was established.           |
| Frontend rendering and URL handling | XSS / unsafe navigation                    | No issue found            | No credible unsafe HTML execution path survived.                                                    |
| Container and Nginx configuration   | Exposure / privilege                       | Needs follow-up           | Operational defects are listed below; no direct high-impact exploit survived.                       |
| Tracked repository content          | Hardcoded secrets                          | No issue found            | High-confidence tracked secret patterns produced no matches; untracked env values were not printed. |

## Code And Operational Errors

These are actionable defects but did not survive the security policy as standalone

vulnerabilities:

1. `backend/Dockerfile:30` executes JSON from `/health` as Python; valid JSON

booleans such as `false` raise `NameError`, so the container health check fails.

1. `frontend/src/lib/api.ts:5` defaults production browser API traffic to `http://localhost:8000`; the production build artifact confirmed that value is embedded. Set `NEXT_PUBLIC_API_URL` at image build time or use a same-origin path.

1. `frontend/src/app/[locale]/settings/page.tsx:49` and `:72` call backend routes (`PUT /api/v1/users/me` and `POST /api/v1/auth/change-password`) that do not exist.

1. Grant-match insertion has no database uniqueness constraint for `(organization_id, grant_id)` and read-then-insert paths can race duplicate rows.

1. The Next.js build warns that the middleware convention is deprecated.  
2. Bandit, pip-audit, and npm audit are non-blocking in CI; fix this release-control debt.  
3. `npm audit` reports two moderate PostCSS advisories through Next.js. The current app path is not exploitable, but dependency upgrade remains appropriate.

1. `pip-audit` reports advisories for `pytest` and `langchain-text-splitters`; the affected LangChain URL-splitting API is not used, and `pytest` should be removed from runtime requirements.

---

## Verification

- Backend: Ruff check passed; 78 pytest tests passed.
- Frontend: lint passed; 24 unit tests across 6 files passed; production build passed.
- Bandit: no medium/high findings; three low-severity observations.
- npm audit: two moderate dependency advisories.
- pip-audit: `pytest` and `langchain-text-splitters` advisories, with no demonstrated request-path exploit.
- Secret review: no tracked high-confidence secret matches.

---

## Open Questions And Follow Up

- Define an explicit role-permission matrix for viewer, writer, and admin, then re-evaluate proposal generation at `backend/app/routers/proposals.py:48-97`.
- Run an isolated, resource-capped parser harness against representative malicious

DOCX and PDF fixtures to measure worker memory, CPU, and retry behavior.

- Verify the actual production Redis URL, TLS/authentication, network policy, and

whether backend port 8000 is directly exposed outside Nginx.

- Repair the production frontend API URL and Docker health check before deployment, even though their strongest consequences are correctness/availability defects.